# CSE 344 – System Programming
# Homework #5
# Priority-Based Cargo Delivery Simulation
## *Due Date: 10.05.2026 – 23:59*

**Important Rules**

• Implemented in C, must compile and run on Debian 12 (64-bit) in VirtualBox.
• Identical or suspiciously similar submissions receive -100 points and are reported for academic misconduct.
• A report with uncut screenshots is mandatory. A submission without a report and makefile is not evaluated.
• Late submissions are NOT accepted.
• Post all questions in the Teams assignment thread so every student benefits from the answers.

---

## 1. Overview

In this homework you will design and implement a Priority-Based Cargo Delivery Simulation using a thread pool architecture. The system reads a list of delivery orders from a file, assigns them to a fixed pool of courier threads based on priority, and produces a structured delivery report.

The system **must use** the following concepts:

- **Thread pool:** a fixed number of courier threads created at startup, never destroyed until shutdown.
- **Priority queue**: orders processed in EXPRESS > STANDARD > ECONOMY order, protecting the shared queue and blocking couriers when idle.
- **Synchronization**: the shared priority queue must be protected and courier threads must block when no order is available.
- **Atomic operations**: lock-free counters for statistics.
- **SIGINT** handling: active deliveries finish, pending orders are cancelled, all threads exit cleanly. SIGINT must be handled safely in a multi-threaded program.

## 2. System Scenario

A cargo distribution center opens its shift every morning. The day's delivery orders are read from a file and placed into a priority queue. A fixed number of couriers begin their shift and wait in the depot. When an order becomes available, the highest-priority order is taken by an available courier. The courier delivers the package and returns to the depot to pick up the next order. When all orders are delivered, the shift ends and a summary is printed.

If a SIGINT signal is received during the shift:

- Couriers currently on a delivery complete it. A delivery cannot be interrupted once started.
- Couriers waiting in the depot do not start new deliveries.
- All pending orders remaining in the queue are cancelled.
- A final summary is printed showing completed and cancelled counts.

## 3. What to Implement

**./cargoGTU -n <num_couriers> -i <orders.txt> -s <stats.txt>**

The program (cargoGTU):

- Reads the delivery orders from the input file at startup.
- Reads all delivery orders from the input file at startup and inserts them into the priority queue before courier threads begin processing.
- Spawns exactly -n courier threads at startup. No new threads are created during the shift.
- Each courier loops: acquire queue_mutex, take highest-priority order, release mutex, deliver, repeat
- Courier threads must block using pthread_cond_wait() when the queue is empty. Busy waiting is strictly forbidden.
- All log lines are printed to stdout protected by log_mutex
- Delivery statistics are written to the file specified by -s after the shift ends
- SIGINT triggers the shutdown sequence described in Section 7

### 3.1 Command-Line Parameters

| Flag | Parameter | Description |
|---|---|---|
| -n | <num_couriers> | Number of courier threads in the pool. Must be >= 1. |
| -i | <orders.txt> | Path to input file. Must exist and be readable. |
| -s | <stats.txt> | Path to statistics output file. |

Missing or invalid parameters must print a usage message to stderr and exit with a non-zero code.

Example: ./cargoGTU -n 5 -i orders.txt -s stats.txt

### 3.2 Atomic Operations

The following counters must be updated using C11 atomic operations (_Atomic / stdatomic.h). **i** Atomic operations guarantee that a single integer read-modify-write is indivisible. They are correct here because each counter update involves only one value. **Using a mutex for these counters will incur a penalty.**

| Counter | Updated when |
|---|---|
| completed_orders | A courier completes a delivery |
| cancelled_orders | An order is cancelled due to SIGINT |
| total_delivery_time | A courier completes a delivery, add duration in ms |

## 4. Input File Format

Each line defines one delivery order:

**<id> <name> <priority> <duration>**

- name: alphanumeric string, no spaces, max 32 characters
- priority: one of EXPRESS, STANDARD, ECONOMY
- duration: positive integer, delivery time in simulation units (1 unit = 500 milliseconds)

Example orders.txt:

```
1 Ahmet_Yilmaz EXPRESS 8
2 Ayse_Kaya ECONOMY 20
3 Mehmet_Demir STANDARD 12
4 Fatma_Sahin EXPRESS 5
5 Ali_Celik STANDARD 9
```

**⚠ Lines with invalid format must be skipped silently. The parser must never crash on malformed input. Blank lines must be skipped.**

### 4.1. Priority Order

| Priority | Level | Description |
|---|---|---|
| EXPRESS | 1 (highest) | Time-critical deliveries, always processed first |
| STANDARD | 2 | Regular deliveries |
| ECONOMY | 3 (lowest) | Non-urgent deliveries, processed last |

When two orders have the same priority, the one with the lower id is processed first.

**⚠ A courier must always take the highest-priority available order. Taking a lower-priority order when a higher-priority one is available is a scheduling error and will be penalized.**

## 5. Outputs

### 5.1 Console Output (stdout)

All output lines are printed to stdout. All lines end with \n. Since multiple courier threads write to stdout concurrently, a dedicated log_mutex must be held before every printf call. Without log_mutex, output lines from different threads will interleave and corrupt each other. Interleaved or corrupted output lines are treated as a synchronization error during grading.

Required output lines and their fields:

| Keyword | Source | Required Fields |
|---|---|---|
| SHIFT_START | [CARGOGTU] | couriers=N orders=M |
| ORDER_QUEUED | [CARGOGTU] | id=X recipient=Y priority=Z duration=W |
| WAITING | [COURIER-N] | (no fields) |
| DELIVERY_START | [COURIER-N] | id=X recipient=Y priority=Z |
| DELIVERY_COMPLETE | [COURIER-N] | id=X recipient=Y duration=Wms |
| SHIFT_OVER | [COURIER-N] | (no fields) |
| SIGINT_RECEIVED | [CARGOGTU] | pending_orders=N |
| ORDER_CANCELLED | [CARGOGTU] | id=X recipient=Y priority=Z |
| SHIFT_END | [CARGOGTU] | completed=X cancelled=Y total_time=Zms |
| SHUTDOWN_COMPLETE | [CARGOGTU] | (no fields) |

Example output:

```
[CARGOGTU] SHIFT_START couriers=3 orders=5
[CARGOGTU] ORDER_QUEUED id=1 recipient=Ahmet_Yilmaz priority=EXPRESS duration=8
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayse_Kaya priority=ECONOMY duration=20
[COURIER-1] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=EXPRESS
[COURIER-2] WAITING
[COURIER-1] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=800ms
[COURIER-1] DELIVERY_START id=2 recipient=Ayse_Kaya priority=ECONOMY
[COURIER-1] DELIVERY_COMPLETE id=2 recipient=Ayse_Kaya duration=2000ms
[COURIER-1] SHIFT_OVER
[COURIER-2] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=5 cancelled=0 total_time=4200ms
[CARGOGTU] SHUTDOWN_COMPLETE
```

**⚠ Output line prefix format is mandatory. [COURIER-N] where N starts from 1. Wrong prefix, missing fields, or interleaved output lines will fail grading checks and indicate a missing log_mutex.**

### 5.2 Statistics File (-s)

Written to the file specified by -s after the shift ends. This is a separate file from stdout — it contains the final numerical summary only, not the per-event output lines. It is written once, after all threads have joined.

Total time is the sum of completed delivery durations, not actual running time. Also, Avg per order = total_time / completed. (If completed is 0, avg per order is 0)

```
SHIFT_SUMMARY
Total orders    : 5
Completed       : 5
Cancelled       : 0
Total time      : 4200ms
Avg per order   : 840ms

COURIER_STATS
Courier-1  completed=2  total_time=2800ms
Courier-2  completed=1  total_time=0ms
Courier-3  completed=2  total_time=1400ms
```

## 6. SIGINT Handling and Clean Shutdown

In a multi-threaded implementation, SIGINT should be handled safely. You implement SIGINT handling using either sigwait() or a signal handler that only sets a flag.

- If sigwait() is used, the main thread directly performs the shutdown sequence.
- If a signal handler is used, it must only set a shared shutdown flag, and all shutdown logic must be handled in normal program flow.

Behavior on SIGINT:

• The program enters shutdown mode.
• Pending orders in the queue are cancelled.
• Waiting courier threads are awakened using pthread_cond_broadcast().
• Couriers that are currently delivering must finish their delivery.
• Couriers that are waiting must not start new deliveries and must exit.
• The main thread waits for all courier threads using pthread_join().
• SHIFT_END and SHUTDOWN_COMPLETE must be printed before exit.
• The statistics file must be written before exit.

## 7. Mandatory Test Scenarios

**Run all five scenarios and include uncut console screenshots in your report**. Each scenario must be run separately and documented with its own screenshot.

Base command for all tests:

```
./cargoGTU -n <N> -i <input_file> -s stats.txt
```

| # | N | Input | What to Verify |
|---|---|---|---|
| 1 | 3 | 10orders_mix.txt | All 10 orders completed. stats.txt written correctly. |
| 2 | 10 | 10orders_economy.txt | All 10 orders are completed without duplication. Since all orders have the same priority, lower id orders must be taken before higher id orders. Multiple couriers should participate when enough work is available. |

| | | | |
|---|---|---|---|
| 3 | 2 | 20orders_mix.txt<br>Send SIGINT after ~2 seconds | Active deliveries complete without interruption. Pending orders cancelled. SHIFT_END shows correct completed + cancelled. (verify with ps after exit). |
| 4 | 3 | 10orders_mix.txt<br>Run under Valgrind | valgrind --leak-check=full: 0 errors, 0 leaks. Show full Valgrind output uncut. |
| 5 | 10 | 20orders_mix.txt<br>Run multiple times | SHIFT_END completed count identical across all runs. No duplicate DELIVERY_START id in stdout. Priority order maintained under high concurrency. |

> ⚠ **All screenshots (section 5.outputs) must be uncut, showing the full console from command prompt through program exit. Cropped or edited screenshots will not be accepted.**

## 8. Submission

Compress as StudentID_HW5.zip. Do not include binaries or .o files or test files.

```
StudentID/
├── Makefile
├── main.c
├── …… (any additional .c / .h files)
└── StudentID_report.pdf
```

Makefile must compile with gcc -Wall -Wextra -std=c11 -pthread and include make clean.
**Do not auto-run inside Makefile.**

## 9. Report Requirements

The report must explain design decisions, not just describe the code, please use diagrams. Pasting code or log output without explanation is not sufficient.

| Section | Content |
|---|---|
| Title Page | Name, ID, course, HW number |
| System Design | Thread pool structure and courier lifecycle. Describe what happens from thread creation to shift end. A diagram is must. |
| Priority Queue Design | How the priority order is enforced in the data structure. |
| Synchronization Strategy | Which primitive protects which resource. |
| SIGINT Handling | Step-by-step |
| Test Scenarios | Uncut screenshots for all 5 mandatory test scenarios, each with a short explanation of what the output shows. |
| Challenges & Solutions | At least 2 concrete problems encountered during implementation and how they were resolved. |

## 10. Grading and Penalties

| Condition | Grading |
|---|---|
| Thread pool: N fixed threads, never recreated during shift | 10 |
| Priority scheduling: EXPRESS > STANDARD > ECONOMY | 15 |
| Shared queue protected, couriers block when idle | 10 |
| Atomic counters: completed, cancelled, total_time via _Atomic | 10 |
| SIGINT and clean shutdown: active delivery finishes, pending cancelled | 10 |
| Console output + statistics file correct | 15 |
| Compilation, Makefile, zero -Wall -Wextra warnings | 10 |
| Report: all 7 sections present, 5 test screenshots uncut | 20 |
| TOTAL | 100 |

**⚠ Grading is holistic. Each criterion on the table assumes the others are correctly implemented. A missing or non-functional component affects the correctness of all criteria that depend on it.**

**Example: if the priority queue is absent, the thread pool cannot be graded for correct scheduling. If log_mutex is missing, output correctness cannot be verified.**

| Condition | Penalty |
|---|---|
| Code does not compile | -100 |
| No report submitted | Not evaluated |
| No Makefile | Not evaluated |
| Late submission | Not accepted |
| Report submitted but screenshots missing or cropped | -80 |
| Missing or broken Makefile | -80 |
| Thread pool not used (new thread created per order) | -80 |
| No real concurrency (single-threaded simulation) | -80 |
| Priority queue absent (orders processed in file order) | -60 |
| Busy waiting / spin-polling anywhere | -60 |
| Same order delivered by two couriers | -40 |
| Delivery interrupted midway on SIGINT | -25 |
| Unjoined or non-terminating courier threads after exit or Ctrl+C | -25 |
| Memory leak detected by Valgrind | -25 |
| log_mutex missing (interleaved output lines) | -20 |

| | |
|---|---|
| Atomic counters implemented with mutex instead of _Atomic | -20 |
| SIGINT handler uses async-signal-unsafe operations | -20 |
| Each -Wall warning (first 10) | -2 each |
| More than 10 -Wall warnings | -25 |